INFIX TO POSTFIX

```
#define <stdio.h>
#define SIZE 50            /* Size of Stack */
#include <ctype.h>
char s[SIZE];
int top = -1; /* Global declarations */

push(char elem) { /* Function for PUSH
operation */
 s[++top] = elem;
}

char pop() { /* Function for POP operation */
 return(s[top--]);
}
int pr(char elem) { /* Function for precedence
*/ switch(elem) {
 case'(':
  return 0;
 case'+':
 case'-':
  return 1;
 case'*':
 case'/':
  return 2;
 }
}main() { /* Main Program */
 char infx[50], pofx[50], ch, elem;
 int i = 0, k = 0;
 printf("\n\nRead the Infix Expression ? ");
 scanf("%s", infx);
 while((ch = infx[i++]) != '\0') {
  if(ch == '(')
   push(ch);
  else if(isalnum(ch))
```

```
   pofx[k++] = ch;
  elseif(ch == ')') {
   while(s[top] != '(')
    pofx[k++] = pop();
   elem = pop(); /* Remove ( */
  } else{ /* Operator */
   while(pr(s[top]) >= pr(ch))
    pofx[k++] = pop();
   push(ch);
  }
 }
 while(s[top] != '/0') /* Pop from stack till
empty */
  pofx[k++] = pop();
  printf("\n\nGiven Infix Expn: %s  Postfix
Expn: %s\n", infx, pofx);
}
```

Postfix Evaluation

```
#include<stdio.h>
#include<ctype.h>
Void main()
{
 float stack[50];
Int i=0, top=-1;
Int val;
Char a[30];
Printf("enter the string");
Scanf("%s", a);
While (a[i]!='\0')
{
If( isalpha(a[i])
```

```
        {
            Printf(" enter the value");
            Scanf("%d", &val);
            Stack[++top]=val;
            Else if(a[i]=='+' II a[i] =='-' II a[i] =='*'
II a[i]=='/')
            {
                If(a[i] =='+')
                {
                    Stack[top-1]=stack[top-1] +
stack[top];
                    Top--;
                    }
i
            If(a[i] =='-')
                {
                    Stack[top-1]=stack[top-1]  -
stack[top];
                    Top--;
                    }
        If(a[i] =='*')
                {
```

```
                    Stack[top-1]=stack[top-1] *
stack[top];
                    Top--;
                    }
If(a[i] ==’/’)
          {
                    Stack[top-1]=stack[top-1] /
stack[top];
                    Top--;
                    }
          i++;
}
Printf(“%f”, stack[top]);
}
```

Most of the bugs in scientific and engineering applications are due to improper usage of precedence order in arithmetic expressions. Thus it is necessary to use an appropriate notation that would evaluate the

expression without taking into account the precedence order and parenthesis.

a) Write a program to convert the given arithmetic expression into

i) Reverse Polish notation(Postfix)

b) Evaluate the above notation with necessary input.

Develop a menu driven program to implement all conversions and evaluations in a single program with necessary inputs